

# **STATUS REPORT 2**

## **REPORT DUE: APRIL 8, 2025**

**Group: 1**

## **1 Introduction**

### **1.1 Highlights**

Our plan for this sprint was to implement as many user interface pages as possible using react, construct a functioning autograder that could successfully grade assignments from 221, and to begin connecting the front and back end so that actions made on the user interface would be met with a response from the back end, such as logging in, creating a class or assignment, submitting an assignment, etc.

### **1.2 Changes**

While developing the autograder, we did not have to use Docker to run student code submissions as expected. Instead, we were able to implement the autograder completely through AWS using S3 buckets and Lambda functions.

## **2 Work Items**

### **2.1 Product Backlog**

At the beginning of our sprint, our product backlog was:

- Create distinct user interfaces for student and instructor views.
- Create a gradebook that supports standards based grading
- Create a submission service that included the use of an autograder
- Set up a basic autograder that connects to Docker containers, finds student code submissions and runs alongside a JSON file the instructor has uploaded with the assignment rubric.
- Host the frontend on a website rather than a local host.
- Organize the backend databases so necessary information is stored and can be easily accessed.
- Require login authentication.
- Develop standards based grading ability
- Create GUI for instructors to create assignment rubric
- Research and implement ways to securely store student data once on a shared server.
- Include review of instructor comments and grading on the student view of assignments.
- Research the potential for Moodle integration of class rosters.

- Allow students to view rubrics for assignments.
- Expand the types of assignments from coding assignments to other file submissions and multiple choice/short response quizzes.
- Meet with professors in the math department to discuss needs of department assignments.

Having a functional user interface and autograder were the priorities communicated by our customer during our last sprint retrospective, and so the sprint backlog we developed for this sprint centered around these goals.

## 2.2 MoSCoW

| Work Item                                             | MoSCoW Categorization | Justification (if needed)                                                                                                                                                                 |
|-------------------------------------------------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Distinct user interfaces for students and instructors | Must have             | <ul style="list-style-type: none"> <li>- Necessary for distinguishing the distinct roles of student and instructor</li> </ul>                                                             |
| Gradebook functionality                               | Must have             | <ul style="list-style-type: none"> <li>- Must have a way to record the autograder grade and instructor grade based on rubric. Additional gradebook features are “Should have.”</li> </ul> |
| Submission service that includes an autograder        | Must have             | <ul style="list-style-type: none"> <li>- This is the main goal of the product, as expressed repeatedly by the customer</li> </ul>                                                         |
| Autograder that provides grading feedback             | Must have             | <ul style="list-style-type: none"> <li>- Grading feedback is necessary to review the autograder’s accuracy and clarify problems in student code</li> </ul>                                |
| Third-party hosting                                   | Should have           | <ul style="list-style-type: none"> <li>- Needed long term to use the website</li> </ul>                                                                                                   |
| Login authentication                                  | Should have           | <ul style="list-style-type: none"> <li>- Needed long term if</li> </ul>                                                                                                                   |

|                                                   |             |                                                                                 |
|---------------------------------------------------|-------------|---------------------------------------------------------------------------------|
|                                                   |             | adopted by Davidson                                                             |
| Support standards-based grading                   | Should have | - Important for expanding functionality beyond autograding assignments          |
| Create assignment rubrics                         | Must have   | - Necessary for grading coding assignment                                       |
| In-line instructor comments on coding assignments | Could have  | - Not integral to the grading of assignments, but would be helpful for students |
| Support quizzes and pdf file submissions          | Should have | - Important for expanding functionality beyond autograding assignments          |
| Moodle integration of class rosters               | Won't have  | - Useful in long term adoption by Davidson                                      |
| Allow students to view rubric for assignment      | Could have  | - Helpful for students                                                          |

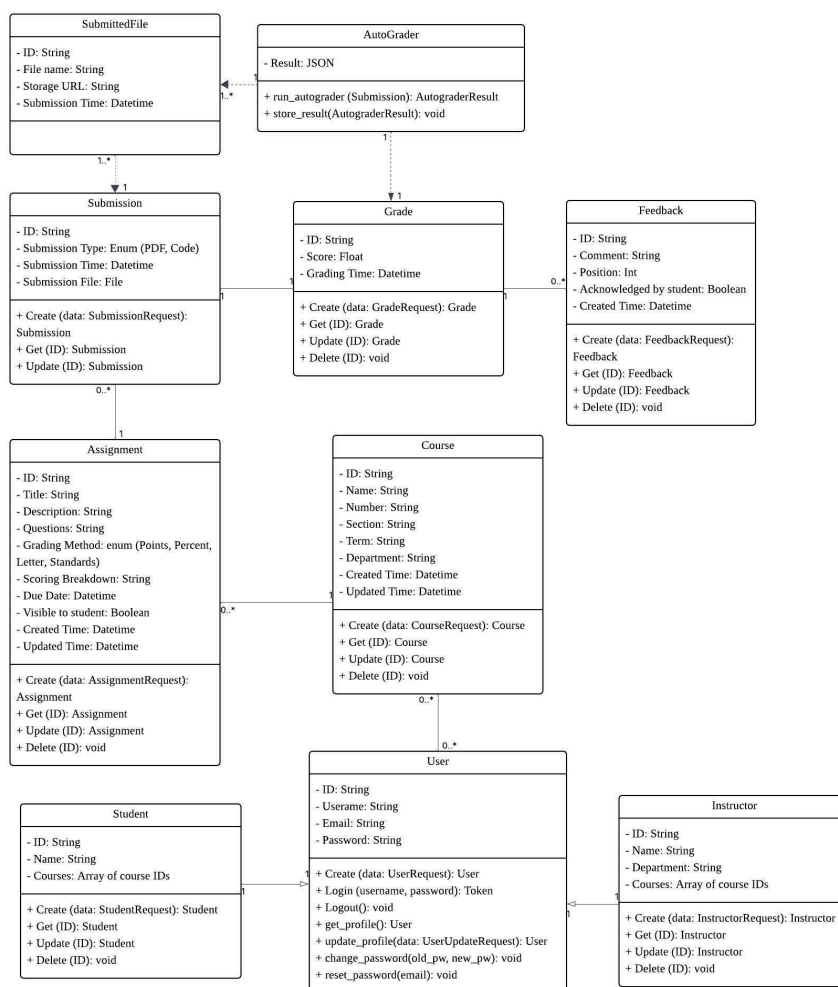
## 2.3 Sprint Backlog

Sprint Backlog:

1. Implement Figma design on web server using react (Must have)
2. Host site on server (Should have)
3. Connect front and back end (Must have)
4. Connect autograder to front and back end (Must have)
5. Update database and user interface to present necessary course data (Must have)
6. Clean up backend code files and API structure to be more consistent, document API endpoints format (Must have)

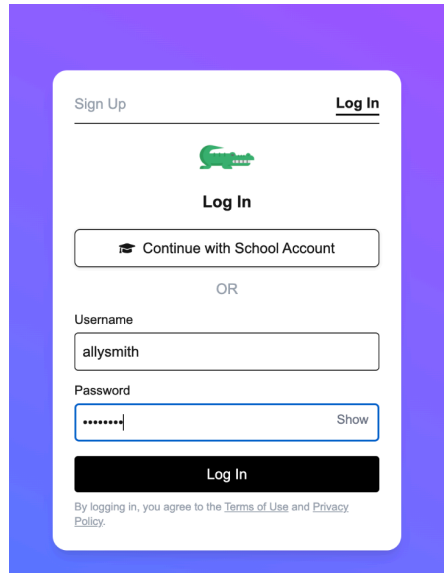
Going into this sprint, our primary goals were to get a basic, but functional, user interface up and working, create an autograder that could successfully run and grade example assignments from CSC 221, and to begin connecting pieces of the front end and back end. We successfully implemented our Figma designs to create a simple user interface. We were also successful in creating an autograder, using AWS, that automatically runs student code submissions when uploaded to the front end, and grades in comparison to a corresponding test case file that professors will upload. We additionally had some housekeeping items on our to-do list like updating the current database scheme to reflect the most up to date understanding of the data we need to collect and to expand upon our documentation, especially for the backend, so that our project can be easily migrated to the next team of developers. Originally, we set out to get our site hosted on a server, instead of locally, but we were not able to complete this task, primarily due to bigger organizational setbacks in trying to coordinate with the University as a whole.

### 3 Unified Modeling Language (UML) – Class Diagram

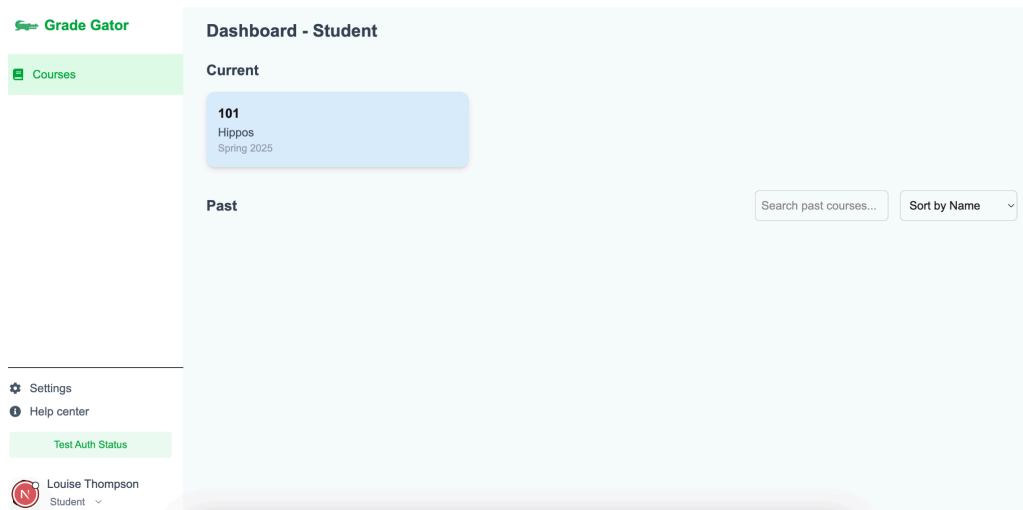


## 4 Current Status

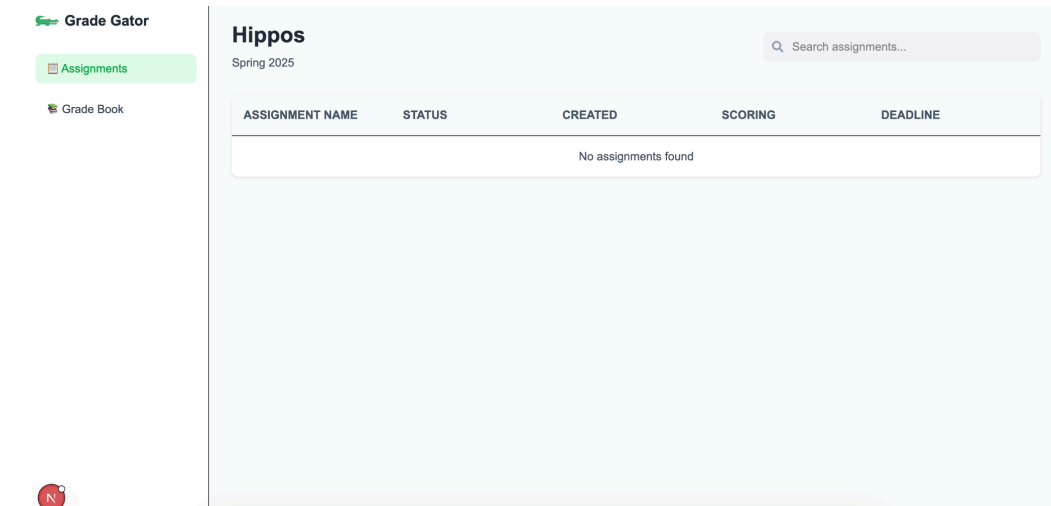
We created superuser accounts in the database, which allows us to login with the superuser credentials on the front end and have our local database data reflected on the front end.

A login form for Grade Gator. At the top, there are links for "Sign Up" and "Log In". Below is a green logo of a gator. The heading "Log In" is centered. A button labeled "Continue with School Account" is present. Below this, the word "OR" is centered. There are input fields for "Username" (containing "allysmith") and "Password" (containing "\*\*\*\*\*"). A "Show" link is next to the password field. A large black "Log In" button is at the bottom. A small disclaimer at the bottom states: "By logging in, you agree to the [Terms of Use](#) and [Privacy Policy](#)."

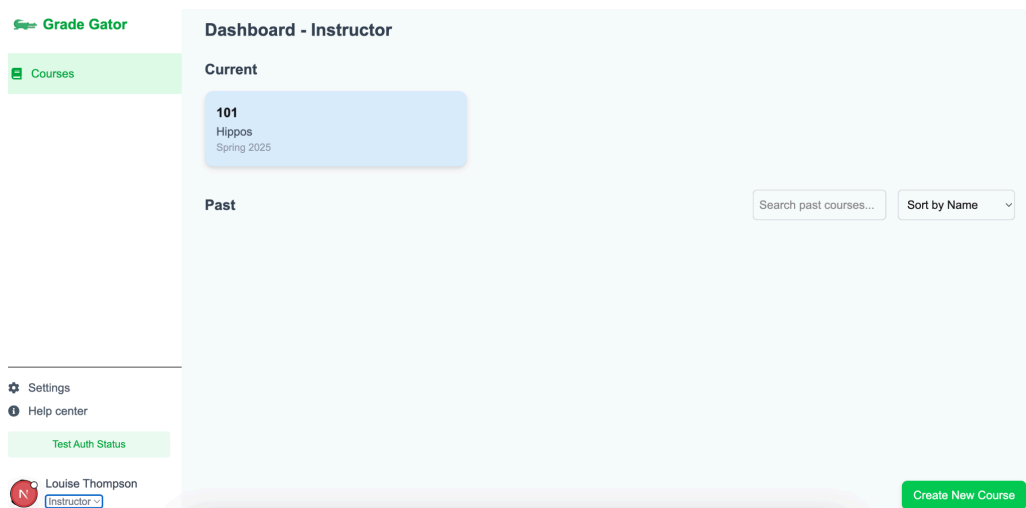
The dashboard in Student view displays all the past and current courses the student is enrolled in. There is search and sorting functionality.

A student dashboard for Grade Gator. The left sidebar has a "Grade Gator" header, a "Courses" button, and a footer with "Settings", "Help center", "Test Auth Status", and a user profile for "Louise Thompson" (Student). The main area is titled "Dashboard - Student". It has a "Current" section with a course card for "101 Hippos Spring 2025". Below is a "Past" section with a search bar "Search past courses..." and a "Sort by Name" dropdown menu.

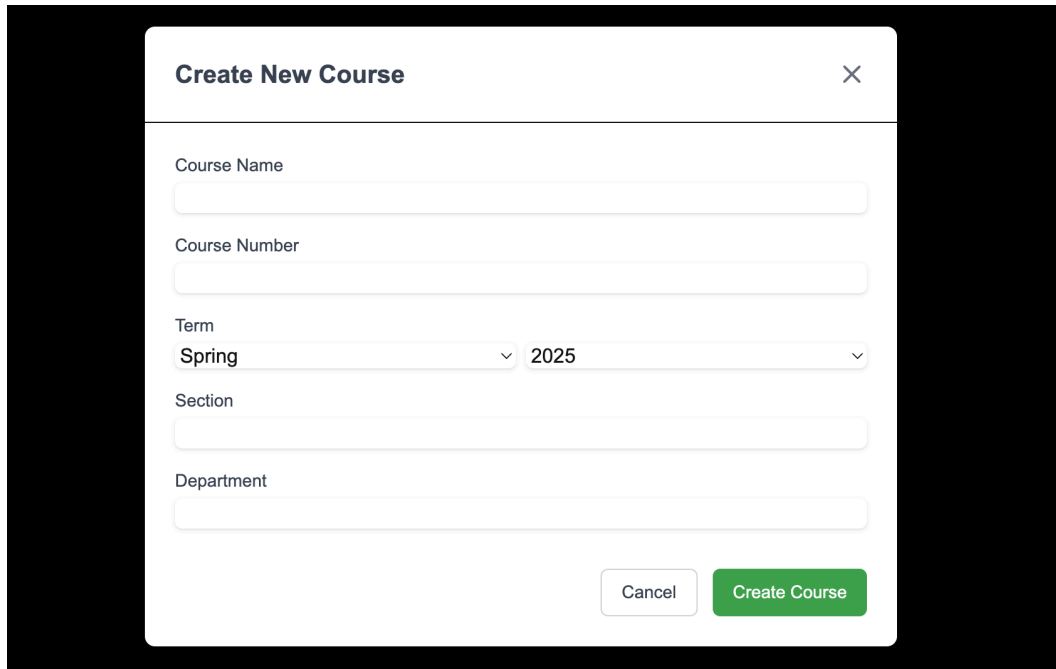
When a student clicks on the course, they are sent to the assignments page for that unique course.



The dashboard with an Instructor view also displays current and past courses, but has an additional “Create New Course” button.



When a user clicks on the button, a popup to create a course comes up asking for information that will be stored in the database and will allow the course to be displayed on the dashboard.

A white modal form titled "Create New Course" with a close button (X) in the top right corner. The form contains five input fields: "Course Name", "Course Number", "Term" (a dropdown menu showing "Spring"), "Section", and "Department" (a dropdown menu showing "2025"). At the bottom right of the form are two buttons: a white "Cancel" button and a green "Create Course" button.

**Create New Course** ✕

Course Name

Course Number

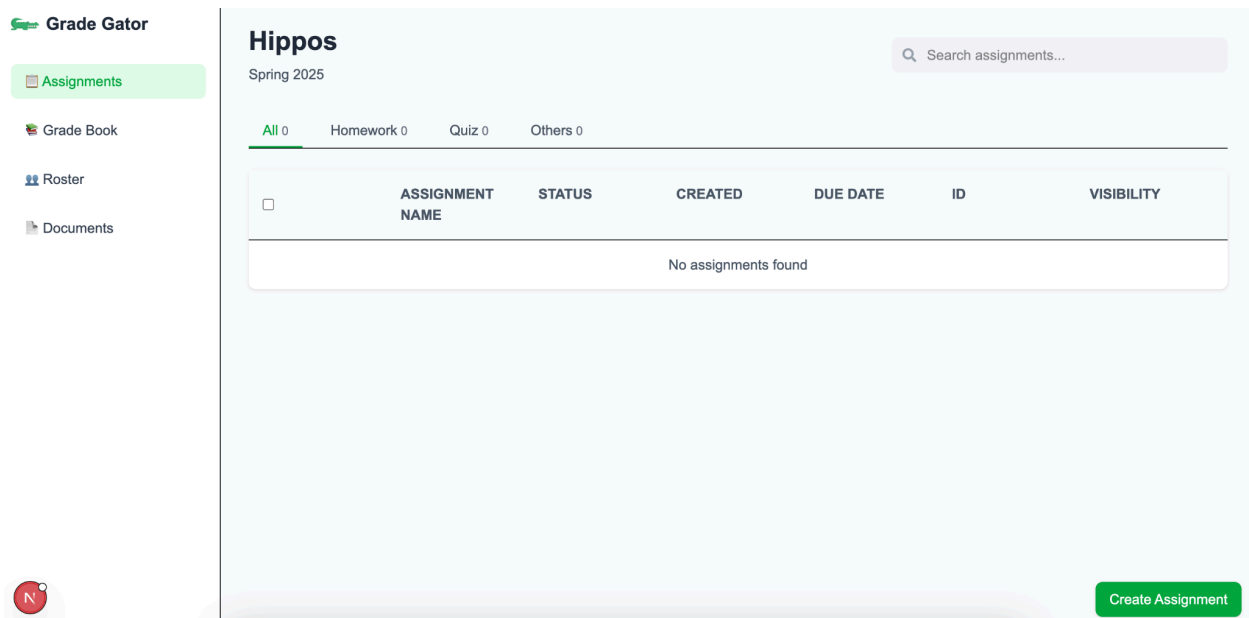
Term  
Spring 2025

Section

Department

Cancel Create Course

When an instructor clicks on the course, they will see the assignments and a “Create Assignment” button.

The interface shows the "Grade Gator" sidebar on the left with options: Assignments (highlighted), Grade Book, Roster, and Documents. The main content area is for a course named "Hippos" in "Spring 2025". It features a search bar "Search assignments...", tabs for "All 0", "Homework 0", "Quiz 0", and "Others 0", and a table with columns: checkbox, ASSIGNMENT NAME, STATUS, CREATED, DUE DATE, ID, and VISIBILITY. The table is currently empty, displaying "No assignments found". A green "Create Assignment" button is located at the bottom right.

**Grade Gator**

- Assignments
- Grade Book
- Roster
- Documents

**Hippos** Spring 2025

Search assignments...

All 0 Homework 0 Quiz 0 Others 0

|                      | ASSIGNMENT NAME | STATUS | CREATED | DUE DATE | ID | VISIBILITY |
|----------------------|-----------------|--------|---------|----------|----|------------|
| No assignments found |                 |        |         |          |    |            |

Create Assignment

An instructor can input information about the assignment, including a rubric. When the instructor hits next, they can upload files for the autograder.

## Create Assignment

Assignment Name \*

☐ Enable anonymous grading

Autograder Points \*

Release Date \*



Due Date \*



☐ Allow late submissions

☒ Enable manual grading

Rubric



+ Add Rubric Item

☐ Enable group submission

Next

## Upload Autograder

Upload a ZIP file to be used as the autograder.

Maximum file size: 10MB

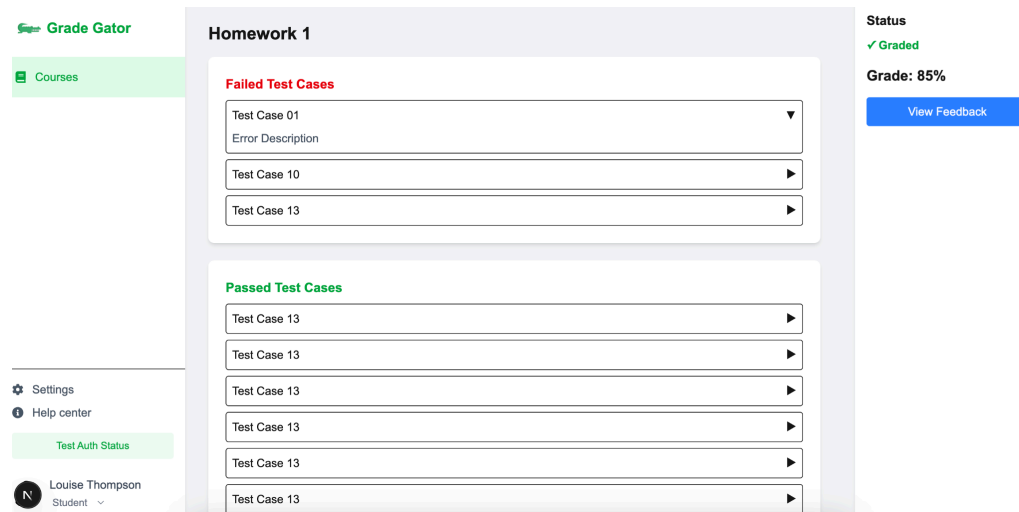
Back

Upload file

Create Assignment



The autograder feedback is currently hardcoded to display the future design. We could not connect the autograder to the frontend due to limitations of personal AWS accounts. Users can see the failed and passed test cases with a drop down to see the error message.



Students can also see instructor feedback, which requires the student to click each one to see the next. They can also click on the feedback and it will highlight the line of code it corresponds to.



Currently, using AWS, we have created a series of S3 buckets, that when working with Lambda functions we've also written, any submissions submitted by a student will enter the ungraded submissions bucket, be automatically taken out and graded against a test case file uploaded by the professor, and then a results JSON file will be generated and put in the graded submissions bucket.

```

1 import json
2 import boto3
3 import os
4 from datetime import datetime
5
6 def lambda_handler(event, context):
7     # Initialize AWS clients
8     s3 = boto3.client('s3')
9     stepfunctions = boto3.client('stepfunctions')
10
11     # State Machine ARN (you'll create this in the next step)
12     state_machine_arn = os.environ['STATE_MACHINE_ARN']
13
14     # Process each message from S3
15     for record in event['Records']:
16         try:
17             # Parse the message body
18             message = json.loads(record['body'])
19
20             # Extract submission details
21             bucket = message['bucket']
22             key = message['key']
23             submission_id = message.get('submission_id', key.split('/')[1])
24
25             # Prepare input for Step Functions
26             step_input = {
27                 'bucket': bucket,
28                 'key': key,
29                 'submission_id': submission_id,
30                 'timestamp': datetime.now().isoformat()
31             }

```

General purpose buckets (4) [Info](#) [All AWS Regions](#) [Copy ARN](#) [Empty](#) [Delete](#) [Create bucket](#)

Buckets are containers for data stored in S3.

Find buckets by name

| Name                                   | AWS Region               | IAM Access Analyzer                         | Creation date                        |
|----------------------------------------|--------------------------|---------------------------------------------|--------------------------------------|
| <a href="#">docker-connection-test</a> | US East (Ohio) us-east-2 | <a href="#">View analyzer for us-east-2</a> | March 23, 2025, 16:21:10 (UTC-04:00) |
| <a href="#">graded-results</a>         | US East (Ohio) us-east-2 | <a href="#">View analyzer for us-east-2</a> | March 31, 2025, 13:57:43 (UTC-04:00) |
| <a href="#">professor-test-cases</a>   | US East (Ohio) us-east-2 | <a href="#">View analyzer for us-east-2</a> | March 27, 2025, 17:16:19 (UTC-04:00) |
| <a href="#">ungraded-submissions</a>   | US East (Ohio) us-east-2 | <a href="#">View analyzer for us-east-2</a> | March 31, 2025, 13:57:55 (UTC-04:00) |

```

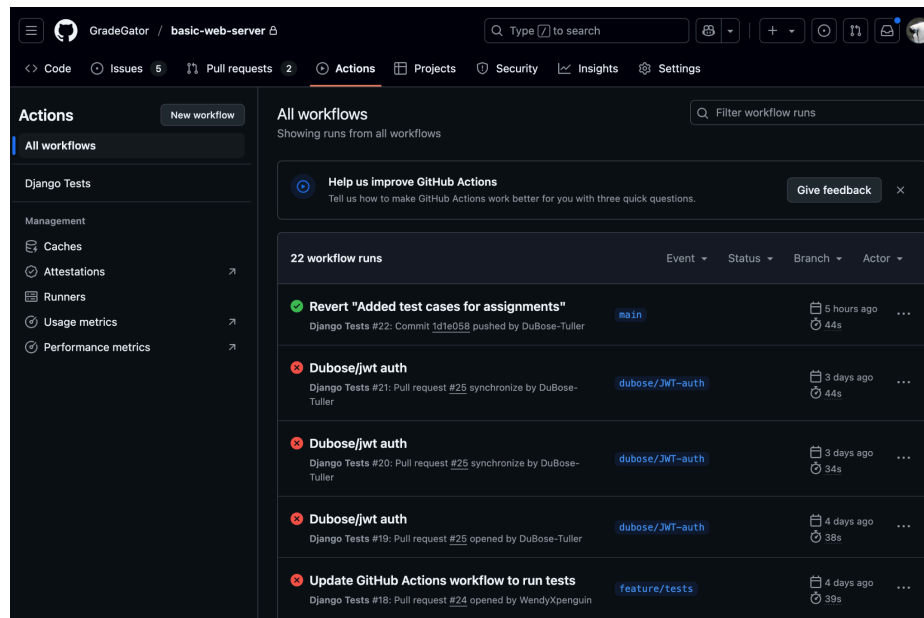
{"status": "success", "output": "Tests run: 3
Tests failed: 0
{
  \"passed\": true,
  \"total\": 3,
  \"failed\": 0,
  \"testResults\": [
    {
      \"testName\": \"testOlympicResult1\",
      \"passed\": true,
      \"input\": \"[ITA 1 0 0, JPN 0 1 0, AUS 0 0 1, KOR 3 1 0, UKR 0 0 1]\",
      \"expectedOutput\": \"[KOR 3 1 0, ITA 1 0 0, JPN 0 1 0, AUS 0 0 1, UKR 0 0 1]\",
      \"actualOutput\": \"[KOR 3 1 0, ITA 1 0 0, JPN 0 1 0, AUS 0 0 1, UKR 0 0 1]\"
    },
    {
      \"testName\": \"testOlympicResult2\",
      \"passed\": true,
      \"input\": \"[AUT 3 4 2, CAN 0 0 1, CRO 0 1 0, FRA 0 1 1, GER 1 1 1, ITA 0 1 1, NOR 1 0 2, SLO 2 0 0, SUI 2 0 1, USA 2 1 2]\",
      \"expectedOutput\": \"[AUT 3 4 2, USA 2 1 2, SUI 2 0 1, SLO 2 0 0, GER 1 1 1, NOR 1 0 2, FRA 0 1 1, ITA 0 1 1, CRO 0 1 0, CAN 0 0 1]\",
      \"actualOutput\": \"[AUT 3 4 2, USA 2 1 2, SUI 2 0 1, SLO 2 0 0, GER 1 1 1, NOR 1 0 2, FRA 0 1 1, ITA 0 1 1, CRO 0 1 0, CAN 0 0 1]\"
    },
    {
      \"testName\": \"testOlympicResult3\",
      \"passed\": true,
      \"input\": \"[AAA 1 0 0, BBB 1 1 0, CCC 1 1 1, DDD 1 1 1, EEF 0 0 0, EEE 0 0 0]\",
      \"expectedOutput\": \"[CCC 1 1 1, DDD 1 1 1, BBB 1 1 0, AAA 1 0 0, EEE 0 0 0, EEF 0 0 0]\",
      \"actualOutput\": \"[CCC 1 1 1, DDD 1 1 1, BBB 1 1 0, AAA 1 0 0, EEE 0 0 0, EEF 0 0 0]\"
    }
  ]
}

```

Test cases can be created based on the functionalities of each module that we want to test. As test cases are created, they can be added to the tests files in each module.

```
tests.py x create_temp_admin.py U ! django.yml ↓ setup.md
assignments > tests.py > ...
1 from django.test import TestCase
2 from django.core.files.uploadedfile import SimpleUploadedFile
3 from django.contrib.auth import get_user_model
4 from courses.models import Course, Student
5 from assignments.models import Assignment, Submission # CORRECT IMPORT
6
7 User = get_user_model()
8
9 class AssignmentModelTests(TestCase):
10     def setUp(self):
11         self.course = Course.objects.create(
12             name="Test Course",
13             number="CS101",
14             term="Fall 2023"
15         )
16
17     def test_assignment_creation(self):
18         """Test that assignments can be created"""
19         assignment = Assignment.objects.create(
20             course=self.course,
21             assignment_id="HW1",
22             title="Test Assignment",
23             description="Test description",
24             grade_method="POINTS",
25             scoring_breakdown="Test breakdown",
26             timing="1 week",
27             due_date="2025-05-01T00:00:00Z",
28             is_visible_to_students=True
29         )
30         self.assertEqual(str(assignment), "Test Assignment (HW1)")
31         self.assertEqual(assignment.course, self.course)
```

Here are some example test cases inside the tests file for the assignment module. As for the superuser access, a function that supports the creation of a temporary superuser is implemented. A better integration of the GitHub and the test cases is still being developed.



## 5 Sprint Review

We met with two of our customers, Dr. Lim and Dr. Pulaj, on Friday, April 4th. They expressed positive feedback about the progress we made. They suggested focusing on adding the functionality of submitting multiple files at once as well as creating more documentation for future developers to work off of. Dr. Lim is actively working with T&I and our group to hopefully gain access to an organization-level AWS account during the next sprint.

## 6 Sprint Retrospective

Katherine, Maddie, and Ally worked on frontend pages that would display the functionality of the backend. They created login, dashboard, course, assignment, autograder and instructor feedback pages. They were not able to connect the autograder to the feedback pages because we need an organization AWS account. Until the web server and database are hosted on AWS, we cannot reliably use a locally hosted frontend to make API calls to the backend that call AWS services. We were, however, able to add API calls to the frontend that would query and/or save data in our local backend databases. We are able to see these changes displayed on the frontend website, but it is all local for the time being.

DuBose updated the backend web server to support token-based authorization. This will lay the groundwork for future authorization mechanisms such as Davidson SSO and AWS Cognito. He also improved the overall structure of the API endpoints.

David, Ann, and Alice worked on configuring the autograder during this sprint. They created S3 buckets that would connect to Lambda functions in AWS and automatically run and grade student submissions when uploaded to the front end.

Wendy worked on creating test cases, testing the test cases locally, and connecting them to GitHub for testing all test cases each time when a new branch is being pushed. The test cases can be added to test files under each module, and by accessing via a superuser identity, one can run the test cases on GitHub.

Team management has not been much of a challenge for us. The biggest challenge is finding a time when all members can meet. However, we have subgroups to complete different tasks. The subgroups can meet more frequently and the whole team can bring the pieces together when we do meet. We communicate often over Slack and keep each other updated on any progress made.

## 7 Goals for the Final Iteration

Product Backlog:

- Create a gradebook that supports standards based grading
- Connect the front end to the autograder

- Develop documentation and to-do list for next set of developers
- Connect creating assignments on front end to back end database
- Enable the submission and grading of assignments with multiple files
- Create PDF submission support for assignments
- Host the frontend on a website rather than a local host.
- Organize the backend databases so necessary information is stored and can be easily accessed.
- Require login authentication.
- Develop standards based grading ability
- Expand GUI for instructors to create assignment rubric
- Research and implement ways to securely store student data once on a shared server.
- Research the potential for Moodle integration of class rosters.
- Allow students to view rubrics for assignments.
- Expand the types of assignments from coding assignments to other file submissions and multiple choice/short response quizzes.
- Meet with professors in the math department to discuss needs of department assignments.
- Add fun gator graphics

#### Sprint Backlog:

1. Connect the front end to the autograder
2. Develop documentation and to-do list for next set of developers
3. Connect creating assignments on the front end to back end database
4. Enable the submission and grading of assignments with multiple files
5. Create PDF submission support for assignments
6. Host the frontend on a website rather than a local host

#### Acceptance Tests:

- User Authentication
  - Given that a user is not logged in, when they visit the login page and enter valid credentials, they should be redirected to the dashboard and see their courses and assignments.
- Course Management
  - Given a logged-in instructor, when they click "Create Course" and fill in the course details (name, description, semester), then a new course should be created and they should be redirected to the course page.
  - Given a logged-in instructor viewing their dashboard, when viewing their courses, then they should see a list of the courses they teach and previously taught

- Given a logged-in student, when viewing their dashboard, then they should see a list of all the courses they are enrolled in
- Assignment Management
  - Given a logged-in instructor viewing a course, when they create a new assignment and they fill in assignment details (name, due date, description, etc.), then the assignment should appear in the course assignments list
  - Given a logged-in student viewing a course, when they click on an assignment, then they should see the assignment details and they should see the submission option if the deadline hasn't passed
- Assignment Submission
  - Given a logged-in student viewing an active assignment, when they upload their submission file and click submit, then the submission should be recorded and returned to the course page
- Grading and Feedback
  - Given a logged-in instructor viewing an assignment, when they view the submissions, then they should see a list of all student submissions
  - Given a logged-in instructor viewing a submission, when they provide feedback and a grade, then the feedback should be saved and the student should be able to view it
  - Given a logged-in student, when view a graded assignment, then they should see their grade and feedback
- Auto-grading
  - Given an assignment with auto-grading enabled, when a student submits their work, then the auto grader should process the submission and provide immediate feedback if configured to do so
  - Given an instructor setting up an assignment. when they upload an auto grader configuration, then the system should validate and accept the configuration